



TRAINING PROGRAMME

Docker & Kubernetes

Package applications in containers, run them on Kubernetes and deliver them with GitOps

57 HOURS

FOR Developers, system administrators and DevOps engineers starting with containers



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This programme takes participants from their first container to a multi-service application running on Kubernetes and delivered by GitOps. They build small, secure images, run multi-container applications with Docker Compose, then deploy them on Kubernetes: workloads, services and network policies, configuration and storage, traffic routing with the Gateway API and TLS, autoscaling, Helm charts, monitoring, security and an overview of managed clusters on EKS and AKS. The last phase introduces GitOps with Argo CD and per-environment configuration with Kustomize and Helm. The same application is carried through every phase, and the programme closes with a final project in which teams containerise, deploy and present a multi-service application.

At a glance

#	PHASE	FOCUS	HOURS
01	Docker	Images, containers, storage, networking, Compose and security	15 h
02	Kubernetes	Workloads, services, configuration, storage, routing, scaling, Helm, security	18 h
03	GitOps	Argo CD, the pull model, environments with Kustomize and Helm	9 h
04	Final Project	A multi-service application, from Compose to GitOps	6 h
	Buffer	Catch-up, revision, extra lab time	9 h
	Total		57 h

Key facts

AUDIENCE	Developers, system administrators and DevOps engineers starting with containers
PREREQUISITES	Linux command line (course: Linux & Bash Scripting, or equivalent). No prior container experience needed
DURATION	57 h: 48 h of chapters + 9 h buffer
FORMAT	Short lectures followed by guided labs; one application carried through every phase
LAB ENVIRONMENT	Runs locally: one laptop per participant (16 GB RAM minimum, Docker, kind, kubectl, Helm, VS Code) and a GitHub repository per team for GitOps; managed clusters (EKS, AKS) shown by trainer demonstration; no cloud account or cost for participants
ASSESSMENT	A lab validated at the end of each phase and a final project presented to the group; certificate of completion for participants who complete both
OUTCOME	Participants can containerise a multi-service application, deploy it on Kubernetes with Helm, routing, TLS, autoscaling and monitoring, and deliver it from Git with Argo CD

PHASE 01

15 H

Docker

Participants learn what a container really is, then build and run them: efficient multi-stage images, persistent storage, container networking and multi-container applications with Docker Compose, with security scanning and the practices that keep containers easy to operate. Validated lab: the course application (frontend, API and database) running with Docker Compose from small, non-root images that pass a Trivy scan.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	Containers vs VMs	Namespaces, cgroups, OCI, Docker Engine, containerd	Lecture + Lab	1.5 h
1.2	Running Containers	run, ps, exec and logs, ports, environment variables, restart policies, resource limits	Lecture + Lab	1.5 h
1.3	Building Images	Dockerfile, layers and cache, CMD vs ENTRYPOINT, multi-stage builds, image size	Lecture + Lab	3 h
1.4	Storage	Volumes, bind mounts, tmpfs, backup and restore	Lecture + Lab	1.5 h
1.5	Networking	Bridge and host networks, overlay networks (concept), DNS between containers	Lecture + Lab	1.5 h
1.6	Docker Compose	Services, health checks, env files, profiles, development and production overrides	Lecture + Lab	3 h
1.7	Security & Registries	Trivy scanning, non-root users, read-only filesystems, private registry	Lecture + Lab	1.5 h
1.8	Best Practices	Twelve-factor apps, logging, sidecar pattern, debugging containers	Lecture + Lab	1.5 h

PHASE 02

18 H

Kubernetes

Participants move the course application from Compose to a local Kubernetes cluster and learn each object it needs: workloads, services and network policies, configuration and secrets, persistent storage, traffic routing with the Gateway API and TLS from cert-manager, resource limits and autoscaling, Helm packaging, monitoring with Prometheus and Grafana, and basic security. Ingress is covered for existing clusters, including migration away from the retired ingress-nginx. Validated lab: the course application installed from a Helm chart, reachable over HTTPS through the Gateway API and scaled by the Horizontal Pod Autoscaler.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Architecture	Control plane, nodes, kubectl, local clusters (kind, minikube)	Lecture + Lab	1.5 h
2.2	Pods & Deployments	Pods, ReplicaSets, Deployments, rolling updates	Lecture + Lab	1.5 h
2.3	Other Workloads	DaemonSets, StatefulSets, Jobs, CronJobs	Lecture + Lab	1.5 h
2.4	Services & Networking	ClusterIP, NodePort, LoadBalancer (cloud-provider-kind locally), CoreDNS, NetworkPolicies	Lecture + Lab	1.5 h
2.5	Configuration	ConfigMaps, Secrets	Lecture + Lab	1.5 h
2.6	Storage	PersistentVolumes, PersistentVolumeClaims, StorageClasses	Lecture + Lab	1.5 h
2.7	Traffic Routing	Gateway API with Envoy Gateway, Ingress (community ingress-nginx retired in March 2026) and migration, TLS with cert-manager	Lecture + Lab	1.5 h
2.8	Scaling	Requests and limits, resource quotas, Horizontal Pod Autoscaler	Lecture + Lab	1.5 h
2.9	Helm	Charts, values, install, upgrade and rollback	Lecture + Lab	1.5 h
2.10	Observability & Troubleshooting	describe, logs and events, metrics-server, Prometheus and Grafana installed with Helm	Lecture + Lab	1.5 h
2.11	Security Basics	RBAC, ServiceAccounts, Pod Security Standards	Lecture + Lab	1.5 h
2.12	Managed Kubernetes	Amazon EKS and Azure AKS: trainer demonstration, then a comparison exercise	Lecture + Workshop	1.5 h

PHASE 03

9 H

GitOps

With GitOps, the cluster pulls its desired state from Git instead of receiving commands. Participants install Argo CD, deliver the course application from a Git repository, roll back by reverting a commit, and manage development and production environments with Kustomize overlays and Helm values. Validated lab: a change merged in Git that reaches the development environment automatically and production after promotion.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	GitOps & Argo CD	Principles, pull model, Argo CD installation	Lecture + Lab	3 h
3.2	Argo CD in Practice	Applications, ApplicationSets, sync, rollback, app-of-apps	Lecture + Lab	3 h
3.3	Environments	Kustomize overlays, Helm values per environment	Lecture + Lab	3 h

PHASE 04

6 H

Final Project

Teams of two or three containerise and deploy a second provided multi-service application, reusing the Dockerfiles, chart and manifests of the phase labs, then present it as they would to an engineering team.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Brief & Build	Read the brief, containerise the frontend, API and database, write the Compose file for development and the Helm chart	Project	3 h
4.2	Deliver & Document	Gateway API routing with TLS, autoscaling, monitoring, delivery by Argo CD; README and architecture diagram	Project	1.5 h
4.3	Demo & Review	Live demo of a change delivered from Git, then a rollback; questions and feedback	Project	1.5 h

FINAL PROJECT

A Multi-Service Application on Kubernetes

A frontend, an API and a database containerised, run with Compose for development, packaged as a Helm chart, routed over HTTPS through the Gateway API, autoscaled and monitored, and delivered to the cluster by Argo CD from Git.

STACK Docker, Docker Compose, Trivy, Kubernetes (kind), Helm, Gateway API (Envoy Gateway), cert-manager, Prometheus, Grafana, Argo CD, Kustomize, Git

DELIVERABLE Git repository with Dockerfiles, Compose file, Helm chart and Argo CD manifests, architecture diagram, live demo